

基于双路召回与多面过滤的小型学术文献检索系统设计

学号：2410668 姓名：吕昊远

1 背景与意义 (Introduction)

信息检索系统是指根据特定的信息需求而建立起来的一种有关信息搜集、加工、存储和检索的程序化系统，其主要目的是为人们提供信息服务 [6]。在当前的学术研究环境中，科研人员常常需要在本地设备上存储大量以 PDF 格式为主的学术文献。然而，随着文献库规模的扩张，传统的基于操作系统的文件搜索机制暴露出显著的局限性。

首先，传统的文件系统仅支持基于文件名的精准字符匹配，无法深入文献的摘要或正文进行全文的内容检索。其次，当研究人员检索意图（如“图像识别”）与文献实际使用的专业术语（如“卷积神经网络”）发生偏差时，传统的布尔检索模型往往会失效，导致召回率极低，用户无法得到所需的信息。同时，面对库中具有多维特征的文献，搜索时往往缺乏有效的过滤（如发表年份等），使得用户难以快速定位所需的核心文献。

因此，设计并实现一个支持模糊匹配、具备语义理解能力，并能够对本地 PDF 文件流进行自动化处理与多面导航的小型学术文献检索系统，具有重要的工程意义与学术价值。本系统旨在平衡检索效率与精度，为科研人员提供一套轻量化、人性化且高效的文献管理解决方案。

2 现状分析 (Related Work)

当前，针对文献管理与检索的系统目前主要分为两类，分别是传统的文献管理软件与在线的大型通用搜索引擎。

传统文献管理软件（如 Mendeley、EndNote 等）虽然在引文管理方面表现出色，但其内置的检索功能大多严重依赖于用户手动录入的元数据（Metadata，如标题、作者、关键词）。由于缺乏深度的全文语义索引机制，当用户进行探索式查询（Exploratory Search）时，这些软件的检索表现往往差强人意。

另一类也就是类似 Elasticsearch 等企业级的搜索引擎 [2, 4]，虽然底层基于成熟的倒排索引与 BM25 算法，能够提供强大的模糊匹配与高并发检索能力，但其系统架构过于庞大，部署与维护成本极高，不适合作为科研人员本地桌面级的轻量化工具。同时，近年来兴起的基于大语言模型的检索增强生成系统（RAG）[3] 虽然语义理解能力极强，但在本地部署中面临着严重的算力消耗与处理延迟等问题。

经过统计估算，论文的参考文献上限一般不会超过 200 篇，而在大部分学术研究中，论文与会议报告的参考文献通常只有 30 篇左右。这样的轻量化场景往往不需要极致的检索性能，足够方便的检索体验才是提升工作效率的关键所在，而这往往又是传统文献管理软件所不具备的。

综上所述，现有产品难以兼顾轻量化部署与全文语义匹配这两项核心需求，在性能与实用性之间难以找到一个迎合大多数人体验的平衡。基于此现状，本设计提出了一种基于双路召回（稀疏倒排 + 稠密向量）与 RRF 排序融合的折中方案。

3 方案设计 (Framework)

本系统主要实现本地文件流处理与高效检索，整体架构分为三大核心模块：**数据处理层**、**检索层**以及**交互与过滤层**。系统的整体架构如下图所示：

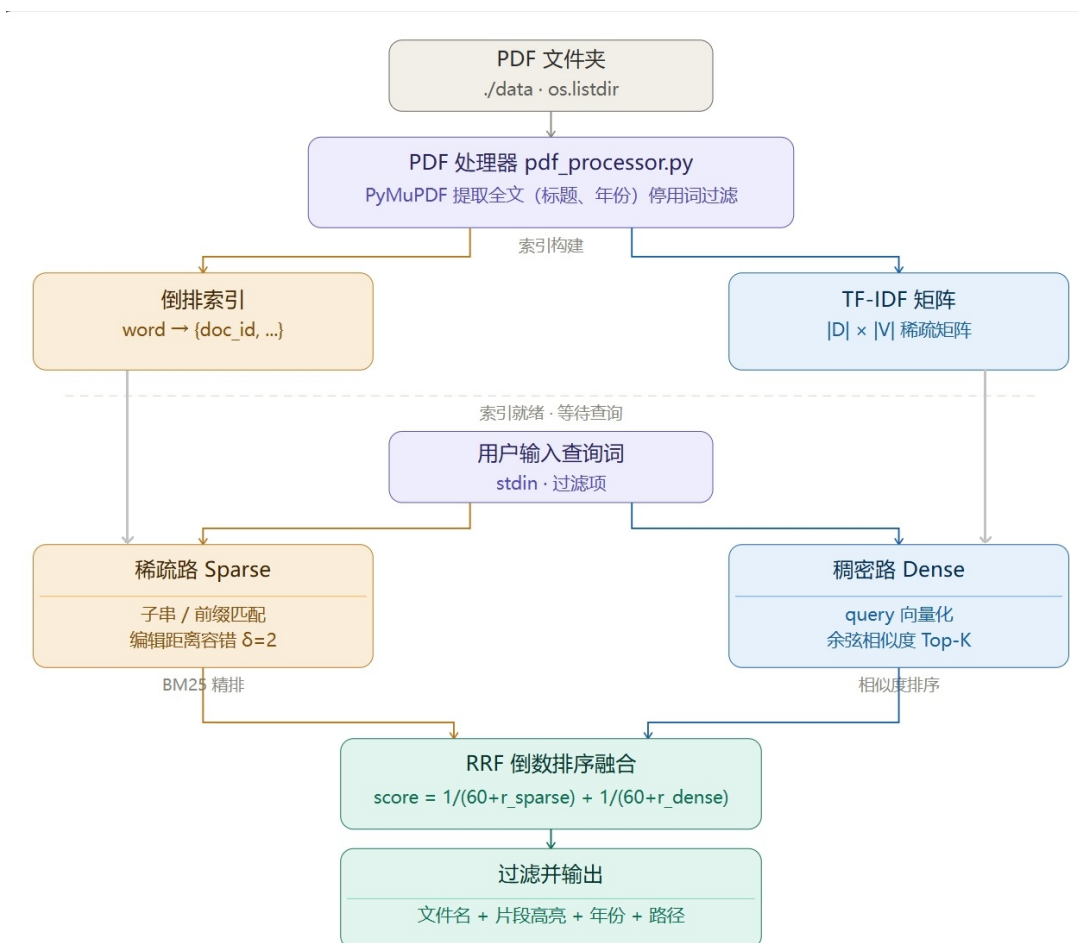


图 1: 系统架构图

在实际使用时，系统启动后首先对指定本地 PDF 文件夹进行文件扫描，完成文档 ID 分配后进入数据处理层，通过 PDF 解析与文本清洗构建倒排索引与 TF-IDF 向量索引，索引就绪后系统进入等待查询状态；当用户输入查询词与过滤条件时，检索层同步启动稀疏路与稠密路双路并行召回，分别完成关键词模糊匹配与语义向量匹配并经各自算法精排，再通过 RRF 倒数排序融合得到全局候选排序结果；最后由交互与过滤层依据用户设定的年份等过滤规则对结果二次筛选，最终输出包含文件信息、命中高亮与路径的检索结果，完成一次完整的文献检索流程。

接下来将分别讲解这三层模块的功能与设计。

1. 数据处理层：系统对指定本地 PDF 文件夹进行扫描，PDF 解析模块提取文档正文文本后，通过正则表达式进行数据清洗与停用词过滤。系统同时采用多策略方案从元数据与正文中抽取标题、发表年份等关键属性，用于后续的多面过滤。

2. 检索层：这部分是系统的核心，由两条独立召回路径并行构成。

- **稀疏路 (Sparse Recall)：**基于倒排索引，支持子串/前缀匹配与编辑距离模糊容错，采用 BM25 算法进行精排打分。

- **稠密路 (Dense Recall)**: 采用 TF-IDF 向量化方案, 将文档与查询分别映射至高维向量空间, 通过余弦相似度进行语义层面的候选文档召回。

两路召回的候选集通过**倒数排序融合 (Reciprocal Rank Fusion, RRF)** 合并为最终排序结果, 兼顾了关键词匹配的精确性与向量检索的语义覆盖。

3. 交互与过滤层: 系统在终端交互中引入多面搜索机制, 允许用户通过设定起始年份等条件对融合排序结果进行二次过滤。最终结果展示包含文件路径直达与包含高亮命中词的上下文片段, 便于用户快速判断文献相关性。

4 技术路线 (Methodology)

为实现上述架构方案, 本项目在技术路线上采取了以下具体策略与算法模型。

4.1 文件流解析与倒排索引构建

系统采用 PyMuPDF 库对本地 PDF 文件进行数据提取。提取后的文本首先改为小写并进行分词处理。在建立倒排索引之前, 系统引入停用词过滤机制, 剔除“the”、“a”、“is”等高频无意义词汇, 以降低索引噪音, 提升精排阶段的打分质量。

最终构建的倒排索引的数据结构表示为字典映射: 键为经过过滤的单词, 值为包含该词的文档 ID 集合。同时记录各文档的词数 (文档长度), 用于后续 BM25 打分中的长度归一化计算。

Listing 1: 倒排索引构建

```
1 STOPWORDS = {'the', 'a', 'an', 'is', 'in', 'of', 'to', 'and', ...}
2 # 停用词过滤
3
4 for idx, doc in enumerate(docs):
5     words = re.findall(r'\w+', doc['content'])
6     filtered = [w for w in words if w not in STOPWORDS and len(w) > 1]
7     self.doc_lengths.append(len(filtered))
8     for word in filtered:
9         self.inverted_index[word].add(idx)
10
11 self.avgdl = sum(self.doc_lengths) / len(self.doc_lengths)
```

4.2 稀疏路: 宽容召回机制

在稀疏路召回阶段, 系统有分为两级的容错匹配策略, 以适应不规范输入的情况:

(1) 子串/前缀匹配: 算法遍历倒排索引键集合, 判定逻辑为 $query \subseteq key$, 可有效处理前缀输入与缩写情形 (如输入“trans”可同时召回“transformer”与“transfer”)。

(2) 编辑距离匹配: 对于未命中子串匹配的索引键, 进一步计算其与查询词之间的**编辑距离** (Levenshtein Distance)。编辑距离定义为将字符串 a 转换为字符串 b 所需的最少单字符操作数 (插入、删除、替换), 这里我们通过动态规划 DP 来求解:

$$d(i, j) = \min \begin{cases} d(i-1, j) + 1 \\ d(i, j-1) + 1 \\ d(i-1, j-1) + 1[a_i \neq b_j] \end{cases} \quad (1)$$

当 $d(\text{query}, \text{key}) \leq \delta$ (本系统默认 $\delta = 2$) 时, 视为模糊命中并将对应文档加入候选集。为降低计算开销, 在调用编辑距离计算前, 系统优先以 $||a| - |b|| \leq \delta$ 进行剪枝。

Listing 2: 容错召回

```

1 def sparse_recall(self, query, edit_threshold=2):
2     q = query.lower()
3     matched = set()
4     for key in self.inverted_index:
5         if q in key: # 子串/前缀: 快速路径
6             matched.update(self.inverted_index[key])
7         elif abs(len(key) - len(q)) <= edit_threshold:
8             if self._edit_distance(q, key) <= edit_threshold:
9                 matched.update(self.inverted_index[key])
10    return matched

```

4.3 BM25 精排

稀疏路在召回阶段后, 采用经典的 BM25 算法 [7] 对候选文档集合进行相关度精排打分:

$$\text{Score}(Q, d) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (2)$$

其中 $k_1 = 1.5$, $b = 0.75$ 为学界常用默认参数; $|d|$ 为文档词数, avgdl 为语料库平均文档词数; IDF 采用 Robertson 平滑公式: $\text{IDF}(q_i) = \log\left(\frac{N - df_i + 0.5}{df_i + 0.5} + 1\right)$ 。

这部分的 BM25 算法主要对候选文档进行了精细化相关度的重排序, 打分结果的将参与到后续 RRF 融合中稀疏侧的排名生成, 为最终的排名提供一定依据。

4.4 稠密路: TF-IDF 向量召回

为弥补纯词符匹配在语义层面的不足, 系统还引入了基于 TF-IDF 的向量化召回路径 [8] 作为另一路的稠密路。文档集合经 `scikit-learn` 的 `TfidfVectorizer` 进行向量化, 采用子线性词频平滑 ($\text{tf} = 1 + \log f_{t,d}$) 以缓解高频词的支配问题, 构建文档-词项矩阵 $\mathbf{M} \in R^{|D| \times |V|}$ 。

对于查询 q , 这里同样映射到该向量空间, 来计算其与各文档向量的**余弦相似度**:

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|} \quad (3)$$

通过余弦相似度的计算, 可以得到映射到向量空间内的夹角, 在一定程度上表征语义之间的关联程度, 在计算完毕后, 相似度大于零的文档会被纳入稠密召回这部分的候选集, 按相似度降序取得其中最为相关的前 K 名 (这里默认使用 $K = 20$)。

4.5 倒数排序融合 RRF

截至到这里, 稀疏路与稠密路已经完成了各自的计算, 两条路径分别从不同角度产生候选文档的局部排名, 接下来将通过**倒数排名融合** (Reciprocal Rank Fusion, RRF) [1] 来将两路结果合并, 从而得到最终的结果。RRF 对每篇文档 d 在各路中的排名 $r_k(d)$ 计算加权倒数之和作为最终融合得分:

$$\text{RRF}(d) = \sum_{k \in \{\text{sparse}, \text{dense}\}} \frac{1}{r_k + \kappa} \quad (4)$$

其中常数 $\kappa = 60$ 为 RRF 原始论文中的推荐值，起到平滑极端排名的作用， k 越大，排名第一和排名第十的差距就越小，融合结果越接近平均。对我们这个小规模场景，即便将 κ 调整到 30 或 80，实际结果差异也不显著，这里我们使用 60 即可。RRF 的优势在于，我们无需对两路得分进行尺度上的归一化，同时其对排名中的异常值也具有较强的鲁棒性。最终，所有候选文档按 $\text{RRF}(d)$ 降序排列，即为系统返回给用户的结果序列，将以一定的格式输出给用户以供使用。

Listing 3: RRF 融合排名

```

1 def dual_recall_and_rank(self, query, rrf_k=60):
2     sparse_set = self.sparse_recall(query)
3     dense_set, d_scores = self.dense_recall(query)
4     all_idx = sparse_set | dense_set
5
6     bm25_ranked = self._bm25_rank(list(all_idx), query)
7     sparse_rank = {idx: r for r, (idx, _) in enumerate(bm25_ranked)}
8
9     dense_sorted = sorted(all_idx, key=lambda i: d_scores[i], reverse=True)
10    dense_rank = {idx: r for r, idx in enumerate(dense_sorted)}
11
12    fused = {}
13    for idx in all_idx:
14        r_s = sparse_rank.get(idx, len(all_idx))
15        r_d = dense_rank.get(idx, len(all_idx))
16        fused[idx] = 1.0 / (rrf_k + r_s) + 1.0 / (rrf_k + r_d)
17
18    return sorted(fused.items(), key=lambda x: x[1], reverse=True)

```

4.6 性能评估对比

对于不同检索机制在效率与精度上的差异特征，在表 1 中进行了理论对比分析。本设计选择了双路融合方案，以在轻量化部署的前提下满足词符匹配与语义匹配两个维度的平衡。

表 1: 不同检索策略的性能评估对比

检索策略	召回速度	匹配精度	容错能力	语义理解
传统布尔检索	极快 ($O(1)$)	高 (字面)	极差	无
纯深度向量检索 (Dense)	慢 ($O(N)$)	极高 (语义)	极强	强
宽容倒排检索 (稀疏路)	快 ($O(K)$)	较高 (BM25)	较强	弱
TF-IDF 向量检索 (稠密路)	较快 ($O(K)$)	较高 (余弦)	较强	中
本设计: 双路 RRF 融合	快	高	强	中

通过对比我们可以发现，单一的检索策略往往存在明显的短板，难以适配实际情况中的多元检索需求。在本设计中选择了融合方案，吸取了不同检索策略之间的优势并进行了合理的平衡，从而使其更能适用于生活中人性化的实际场景。

5 总结 (Conclusion)

本次系统设计主要聚焦于科研人员在本地文献管理与搜索过程中的实际体验。通过引入文件流动态解析、基于倒排索引的前后缀模糊匹配机制以及年份过滤等功能，本方案成功实现了一个轻量、高效且具备较强宽容度的信息检索系统雏形。

本设计中主要展示系统的实现逻辑，在实际应用时，可以考虑对其进行界面的可视化处理。在设计 GUI 时还可以探索出许多实用的小功能，例如将输出的文件位置改为直接把文件展示在界面上便于查看等。

在系统架构上，我们刻意避免了盲目堆砌大型参数模型，将 TF-IDF 向量化与倒排索引这两项在工程上均成熟可靠的检索路径加以组合，通过 RRF 排名融合在尽量不增加系统复杂度的前提下，同时覆盖词符精确匹配与向量语义召回两个维度，实现了效率与精度之间的合理权衡。结果输出时附带的命中词高亮上下文片段也有效提升了用户对检索结果的快速判断能力。

未来，该系统还有许多可以扩展的方向，可以将 TF-IDF 向量替换为轻量级预训练语义向量模型（如 all-MiniLM-L6-v2），以获取更深层的语义理解能力；或者引入轻量向量数据库支持 ANN（近似最近邻）[5] 检索，以在文献库规模扩大时维持检索效率，进一步提升用户体验。

参考文献

- [1] G.V. Cormack, C.L.A. Clarke, and S. Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [2] Elastic. Elasticsearch, 2010.
- [3] Y. Gao, Y. Xiong, X. Liu, et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [4] C. Gormley and Z. Tong. *Elasticsearch 权威指南*. 人民邮电出版社, 北京, 2016.
- [5] Y.A. Malkov and D.A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4):824–836, 2020.
- [6] C.D. Manning, P. Raghavan, and H. Schütze. *信息检索导论*. 人民邮电出版社, 北京, 2019.
- [7] S. Robertson and H. Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [8] G. Salton and C. Buckley. Term weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.